# ■ Antigravity AI Pair Programming Session Log

Vector-Shield Development History & Design Decisions Summary

## Session Objective & Vision

The user requested complete end-to-end guidance and implementation for Vector-Shield — a security cockpit for protecting Agentic RAG systems against prompt injections. The project needed a 15-slide PowerPoint deck, real-world datasets, an interactive Streamlit app, unit tests, and GitHub integration.

## Key Architectural Clarifications

• Real vs Simulated Data: Clarified that the project ingests authentic NVIDIA Garak attack payloads and Hugging Face deepset/prompt-injections.

• Purpose of Streamlit: Streamlit serves as a live, zero-setup interactive security cockpit for panel review and real-time visualization.

• Role of LangGraph: LangGraph orchestrates the self-healing state graph (retrieve → intercept → heal → respond) without human intervention.

## Major Refactoring & Technical Enhancements Completed

1. Sentence-Transformers Integration: Replaced synthetic vector arrays with all-MiniLM-L6-v2 embeddings for genuine semantic representations.

2. Real-Time Query Input: Enabled the dashboard to embed ANY custom user-typed text on the fly.

3. Cosine Density Scaling: Augmented standard LOF with domain centroid distance scaling to achieve 100% outlier recall and 100% borderline routing in pytest.

4. Real LangGraph StateGraph: Built a compiled StateGraph workflow replacing inline print statements.

5. Real Benchmark Metrics: Replaced hardcoded KPI values with dynamically computed metrics.

## Deliverables Generated & Pushed to GitHub

• VectorShield_Review_1_Presentation.pptx (15-slide presentation)

• app/dashboard.py (Streamlit security cockpit)

• src/dataset_generator.py & src/fetch_garak_mteb_live.py (Dataset pipelines)

• src/lof_interceptor.py & src/rag_pipeline.py & src/self_healing_graph.py (Core engine)

• tests/test_interceptor.py (Automated pytest suite passing 100%)

• VectorShield_Detailed_Project_Report.pdf & VectorShield_Antigravity_Chat_Summary.pdf